

Graph-Based Detection of Cooperative Dark Fleets

Full Presentation: From Data to Results to Interpretation

1. Introduction & Goal

1.1 Motivation

Illegal, unreported, and unregulated (IUU) fishing often involves vessels that coordinate at sea: transshipment (transferring catch between vessels), shared fishing grounds, or rendezvous with “dark” vessels that turn off their AIS. Detecting these patterns from publicly available AIS data could help enforcement and research.

1.2 Goal

Find pairs of vessels that may be coordinating or meeting at sea (e.g., transshipment, shared fishing grounds), including when some activity is hidden (e.g., AIS turned off). **We do not have labels for “cooperative” pairs**—this is unsupervised. We use a link-prediction approach: the model learns which vessel pairs tend to co-occur, then we validate high-scoring candidates against raw proximity data. We **surface candidates** with proximity evidence; we do not claim to “detect” or “prove” cooperation.

1.3 Approach (High Level)

1. **Data:** Daily vessel positions (AIS) plus sea-surface temperature (SST)
 2. **Graph:** Build a temporal network where edges = vessels close on the same day
 3. **Model:** Train a Temporal Graph Convolutional Network (TGCN) to learn co-movement patterns
 4. **Ranking:** Score candidate “missing links” (pairs not in the training graph) by embedding similarity
 5. **Validation:** Check whether high-scoring pairs were actually close in the raw data using multiple distance/time thresholds
-

2. Data Pipeline

2.1 Data Sources

- **Vessel positions:** Daily AIS data (latitude, longitude) from 2012-2019. Each vessel has a 9-digit MMSI (Maritime Mobile Service Identity).
- **Environmental:** Sea-surface temperature (SST) from gridded products (GLORYS, Copernicus) - used as optional node features and for QC.

2.2 Pipeline Steps (Explained)

1. **Combine:** Stream raw daily CSVs into a single file (`data/combined_fleet_daily_full.csv`). We use chunked reading to avoid loading ~2 billion rows into memory at once.
2. **Split by year:** Write per-year files so we can parallelize SST extraction and resume if interrupted.

3. **Extract SST:** For each vessel-day position, look up the nearest SST value from the gridded product. We use vectorized operations and a nearest-time/nearest-grid fallback. Output: Parquet chunks under `data/sst_by_year/<year>/`.
4. **QC:** Run `scripts/status_report.py` to count rows with vs. without SST, and compute summary statistics (min, mean, approximate median). This tells us data quality before modeling.
5. **Build graph:** From the combined or SST-enriched data, construct edges where two vessels were within a distance threshold (e.g., 10 km) on the same day. Group edges by time bucket for the temporal model.

2.3 Data Scale

- Total rows: ~2.38 billion vessel-day records
 - SST present: ~2.32 billion (~97.5%); missing ~60 million (~2.5%)
 - Years: 2012-2019
 - The scale motivates sampling (capped rows per day, sampled files per year) for tractability; we also run a full-coverage baseline where resources allow.
-

3. Methods

3.1 Graph Construction (Detailed)

- **Nodes:** Each unique MMSI is a node. We have hundreds of thousands of vessels.
- **Edges:** An undirected edge connects two vessels if they were observed within a distance threshold (e.g., 10 km) on the same day. This captures “who was near whom, when.”
- **Temporal structure:** Edges are grouped into daily time buckets. The model sees a sequence of graph snapshots, one per day.
- **Sampling:** To keep computation tractable, we sometimes cap the number of rows per day and sample a fixed number of daily files per year. The full-coverage run uses all available edges (876 time buckets over 2012-2019).

3.2 Model: TGCN + Temporal Node Features (Explained)

- **TGCN (Temporal Graph Convolutional Network):** A recurrent model that processes the graph snapshot-by-snapshot. At each time step, it updates node representations using both the current graph structure and the previous hidden state. This lets it learn temporal patterns (e.g., vessels that repeatedly co-occur over time).
- **Node features:** We use temporal interaction features from the training period: how many interactions each vessel had, how many unique partners, how recently it was last seen, and the mean gap between interactions. These are concatenated with node degree and fed into the model.
- **Task:** Link prediction. We train on positive edges (pairs that co-occurred) and negative edges (random non-edges). At test time, we score candidate non-edges by the dot-product of their learned embeddings. Higher score = model thinks the pair is likely to be linked.

3.3 Evaluation (Explained)

- **Time-split:** We sort all time buckets chronologically, use the first 70% for training and the last 30% for testing. This is harder than a random split because the model must generalize to future time periods - it cannot “cheat” by using future structure.

- **Metrics:** ROC AUC (area under the receiver-operating-characteristic curve) and Average Precision (AP). Both measure how well the model ranks true edges above false ones. Higher is better; 0.5 is random chance.
- **Multi-seed:** We run with 3 different random seeds and report mean and standard deviation to assess stability.

3.4 Mathematical Formulation

Distance: Haversine Formula

To decide if two vessels are “close” on a given day, we compute the great-circle distance between their positions. The Haversine formula gives the distance in km:

$$d = 2R \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right)$$

where ϕ = latitude (radians), λ = longitude (radians), $\Delta\phi = \phi_2 - \phi_1$, $\Delta\lambda = \lambda_2 - \lambda_1$, and $R \approx 6371$ km (Earth’s radius). **In words:** We convert lat/lon to radians, compute the angular distance using spherical geometry, then multiply by Earth’s radius to get km.

Graph Construction

- **Edge exists** between vessels u and v at time t if: $d(\text{pos}_u(t), \text{pos}_v(t)) \leq \tau$ (e.g., $\tau = 10$ km).
- **Adjacency:** $A_{uv}^{(t)} = 1$ if edge exists, else 0. We use the symmetric (undirected) adjacency.

TGCN: Spatial + Temporal

The TGCN combines a **Graph Convolution** (spatial) with a **GRU** (temporal). At each time step t :

1. **Graph convolution** aggregates neighbor information into each node:

$$f(X_t, A) = \sigma(\tilde{A} \cdot \text{ReLU}(\tilde{A}X_tW_0)W_1)$$

where $\tilde{A} = D^{-1/2}(A + I)D^{-1/2}$ is the normalized adjacency (with self-loops), X_t is the node feature matrix at time t , and W_0, W_1 are learnable weights. **In words:** Each node’s representation is a weighted average of its neighbors’ features plus its own.

2. **GRU update** blends the new graph output with the previous hidden state:

$$\begin{aligned} z_t &= \sigma(W_z[f(X_t, A), h_{t-1}] + b_z) \quad (\text{update gate}) \\ r_t &= \sigma(W_r[f(X_t, A), h_{t-1}] + b_r) \quad (\text{reset gate}) \\ \tilde{h}_t &= \tanh(W_h[f(X_t, A), r_t \odot h_{t-1}] + b_h) \quad (\text{candidate}) \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (\text{new hidden state}) \end{aligned}$$

Here $\odot$ is element-wise multiplication. **In words:** The GRU decides how much to “remember” from the past (h_{t-1}) vs. incorporate the new graph snapshot. This lets the model learn patterns like “vessels that repeatedly co-occur over time.”

Link Prediction Score

For a candidate pair (u, v) not in the training graph, we score it by the **dot product** of their learned embeddings:

$$\text{score}(u, v) = h_u^\top h_v = \sum_{k=1}^d h_{u,k} \cdot h_{v,k}$$

where $h_u, h_v \in \mathbb{R}^d$ are the final hidden states (embeddings) from the TGCN. **Higher score** = model thinks the pair is more likely to be linked (co-occur).

Training Loss (BCE)

We train with binary cross-entropy on positive (real edges) vs. negative (random non-edges):

$$\mathcal{L} = -\frac{1}{N} \sum_i [y_i \log \sigma(s_i) + (1 - y_i) \log(1 - \sigma(s_i))]$$

where $y_i \in \{0, 1\}$ is the label, s_i is the score for pair i , and σ is the sigmoid. **In words:** We push scores of real edges toward 1 and scores of random non-edges toward 0.

Evaluation Metrics

- **ROC AUC:** Area under the curve of True Positive Rate vs. False Positive Rate as we vary the score threshold. AUC = 0.5 means random; AUC = 1.0 means perfect ranking.
- **Average Precision (AP):** Area under the precision-recall curve. Emphasizes how well we rank the few true positives at the top.

Temporal Node Features (Summary)

For each vessel v in the training period: *interactions_count* = number of edges, *unique_partners* = number of distinct neighbors, *last_seen_days* = days since last edge, *mean_gap_days* = mean interval between edges. These are standardized and concatenated with node degree as input X_t .

4. Results

4.1 Baseline Performance (Explained)

Setting	Model	ROC AUC	AP
Full coverage (2012-2019)	TGCN + temporal features	~ 0.679	~ 0.695
Full coverage (1500 train buckets)	TGCN + temporal, emb32, ep5, lr0.001	~ 0.78	~ 0.80
Capped graph (reproducible)	TGCN + temporal features	~ 0.95	~ 0.97
Capped graph (tuned)	TGCN + temporal, emb32, ep5, lr0.001	~ 0.995	~ 0.99

Setting	Model	ROC AUC	AP
Capped graph (5-fold CV)	TGCN + temporal, emb32, ep5, lr0.001	0.927 ± 0.067	0.947 ± 0.048

Why the difference? The full-coverage run uses all 876 time buckets and many more edges - it is a harder, more realistic setting. Using `--max-train-buckets 1500` (Mac-friendly) yields ROC AUC ~ 0.78 , improving over the original. The capped graph uses fewer edges and a smaller graph, so the model achieves higher metrics. Hyperparameter tuning on the capped graph yields ROC AUC ~ 0.995 (best: embedding_dim=32, epochs=5, lr=0.001). We report both: the full-coverage result as the main benchmark, and the capped result to show the pipeline works and is reproducible on typical hardware.

Key finding: Time-split evaluation is substantially harder than random split. Earlier experiments with random splits reached ROC AUC ~ 0.93 ; time-split drops to ~ 0.68 . Temporal features and temporal GNNs help compared to static baselines.

Cross-validation: Use `--cv-folds 5` for rolling-window CV: each fold trains on an expanding window (50%–90% of buckets) and tests on the immediate future. Capped graph: ROC AUC 0.927 ± 0.067 , AP 0.947 ± 0.048 across 5 folds.

4.2 Candidate Ranking (Explained)

- **Output:** We sample candidate non-edges (pairs not in the training graph), score them with the trained model, and rank by score. The top-1000 are saved to `artifacts/tgcn_candidate_scores_fullcoverage.parquet` (1500-bucket model).
- **Score interpretation:** The top-1000 scores are tightly clustered (mean ~ 30.5 , std ~ 0.24 for the 1500-bucket model). We use the top-50/100 for manual review and the top-500/1000 for broader pattern analysis.

5. Validation: Multi-Threshold Cross-Check (Explained)

The model ranks pairs by embedding similarity, but that does not guarantee they were actually close in the raw data. Many high-scoring pairs may simply fish in the same broad region. We therefore validate by checking: for each high-scoring pair, how many days were both vessels within a given distance, allowing a time window?

Threshold	Meaning	Pairs with ≥ 1 close day
25 km +-1 day	Both vessels within 25 km, same day or +-1 day	5
50 km +-3 days	Both within 50 km, within 3 days	8
100 km +-7 days	Both within 100 km, within 7 days	11

Why multiple thresholds? Stricter thresholds (25 km) are stronger evidence of real proximity. Looser thresholds (100 km) catch more pairs but include “same region” overlap. If a pair passes 25 km, it almost always passes 50 km and 100 km - that consistency suggests the proximity is real, not a coincidence. Pairs that only pass 100 km may be regionally aligned without actually meeting.

Why so few pairs? We only validate the top-100 model-ranked pairs. Of those, only 5 have any days within 25 km. Most high-scoring pairs are “same fishing grounds” without close encounters - the validation filters those out.

6. Case Studies: Top 4 Validated Pairs

Summary Table

src	dst	score	within 25km +-1d	within 50km +-3d	within 100km +-7d
412000690	412325200	15.92	102	410	1568
412061791	412508302	15.81	8	88	378
412425192	998508450	14.86	9	20	53
978925333	415131223	14.80	2	12	864

Pair 1: 412000690 <-> 412325200 (Strongest)

- **102 days within 25 km +-1 day** - the strongest validation signal among all pairs. This means on 102 separate days (allowing +-1 day), the two vessels were within 25 km of each other.
- Mean distance when both active: ~135-139 km across thresholds - indicating sustained co-presence in the same operating area rather than a single encounter.
- Overlap of 774 days (25 km window) and 1,767 days (50 km window) suggests multi-year, repeated close encounters.
- **Interpretation:** Highest-confidence candidate for cooperative or coordinated behavior. Worth external metadata lookup (flag, gear type, AIS history) to validate. Use as the primary case study in the write-up.

Pair 2: 412061791 <-> 412508302 (Moderate)

- 8 days within 25 km; 88 within 50 km; 378 within 100 km. Lower counts than Pair 1.
- Mean distance ~216 km (25 km window) to ~252 km (100 km window).
- **Interpretation:** Possible transshipment rendezvous or shared fishing grounds. Weaker evidence than Pair 1 but still above chance. Good secondary case study.

Pair 3: 412425192 <-> 998508450 (Small but non-zero)

- 9 days within 25 km; 20 within 50 km; 53 within 100 km.
- Overlap of 140 days (25 km) and 355 days (50 km) - fewer total encounters than Pairs 1 and 2.
- **Interpretation:** Could indicate occasional coordination or shared port/region. Useful for geographic diversity in the case-study set.

Pair 4: 978925333 <-> 415131223 (Contrast)

- Very low at 25 km (2 days); 12 at 50 km; 864 at 100 km. High overlap at 100 km but few close-proximity days at tight thresholds.

- **Interpretation:** Likely regional co-activity (same fishing grounds or migration corridor) rather than direct side-by-side coordination. This pair illustrates the value of multi-threshold checks: the model surfaces it, but validation shows it is “same region” not “actually close.”

7. Track Plots: Interpretation Guide

The plots show **daily mean latitude/longitude** for each vessel in a pair over time. Each colored line is one vessel’s trajectory; points are daily positions.

What to look for

- **Overlapping or parallel tracks:** Vessels operating in the same area over time. Suggests shared fishing grounds or migration corridor.
- **Converging segments:** Tracks that come together - potential close encounters or rendezvous.
- **Sustained co-location:** Tracks that stay close for extended periods - stronger evidence of coordination.

How to read the axes

- X-axis: Longitude (degrees East)
- Y-axis: Latitude (degrees North)
- Time flows along the trajectory; earlier points connect to later ones along each line.

Plot 1: 412000690 <-> 412325200

Pair 412000690 and 412325200

Figure 1: Pair 412000690 and 412325200

Strongest candidate: 102 days within 25 km. The tracks show sustained co-presence - both vessels operate in overlapping regions over multiple years. Converging segments indicate repeated close encounters.

Plot 2: 412061791 <-> 412508302

Pair 412061791 and 412508302

Figure 2: Pair 412061791 and 412508302

Moderate validation: 8 days within 25 km. Tracks show some overlap with intermittent close proximity - consistent with occasional rendezvous rather than persistent co-location.

Plot 3: 412425192 <-> 998508450

Small but non-zero close-proximity days. Tracks may show occasional overlap; useful for geographic diversity in the case-study set.

Pair 412425192 and 998508450

Figure 3: Pair 412425192 and 998508450

Pair 978925333 and 415131223

Figure 4: Pair 978925333 and 415131223

Plot 4: 978925333 <-> 415131223

Contrast case: 2 days within 25 km but 864 within 100 km. Tracks likely show regional overlap (same fishing grounds) rather than direct close encounters.

8. Limitations (Explained)

- **Grid-cell resolution:** Positions are stored as daily grid-cell centroids, not exact GPS. Fine-scale transshipment (e.g., vessels meeting within a few km for a few hours) may be blurred or missed. We cannot distinguish “same cell” from “adjacent cells” at sub-daily resolution.
- **No ground truth:** We have no labeled “cooperative” or “bridge-vessel” pairs. Validation is based on proximity heuristics - we assume that repeated close proximity is a proxy for coordination. We cannot confirm actual behavior (transshipment, etc.) without external sources.
- **MMSI validity:** A small number of candidate IDs (e.g., under 100M) may be non-standard or corrupted; we filter these in reporting.
- **Temporal coverage:** Results depend on the 2012-2019 slice and the specific edge-construction parameters (distance threshold, time bucket, sampling). Different parameters could yield different candidate sets.

9. Conclusion

- The TGCN + temporal features model surfaces candidate vessel pairs that have observable proximity evidence in the raw data.
- Pair 1 (412000690 <-> 412325200) has the strongest validation (102 days within 25 km) and is the primary case study for “model surfaces plausible bridge-vessel pairs.”
- Multi-threshold validation separates “same region” overlap from “actually close” encounters. Pairs that pass 25 km are more credible than those that only pass 100 km.
- The pipeline is reproducible. The full-coverage run requires substantial RAM; the capped graph runs on typical laptops and validates the approach.

10. Future Work

- **Finer resolution:** Sub-daily or higher spatial resolution would improve detection of brief transshipment events.

- **External metadata:** Flag, gear type, ownership, and AIS history from vessel registries would strengthen case-study interpretation.
- **Supervised labels:** If labeled “cooperative” pairs become available (e.g., from enforcement), a supervised model could be trained.
- **Full-coverage tuning:** Hyperparameter tuning on the capped graph yields ROC AUC ~0.99; applying the best config (embedding_dim=32, epochs=5, lr=0.001) to full-coverage when resources allow.
- **Ensemble scoring:** Averaging scores over multiple seeds improves ranking stability.

Appendix: Key Commands

```
# Build temporal graph
python3 scripts/build_temporal_graph_baseline.py --years 2012,2013,2014,2015,2016,2017,2018,2019 --out-  
  
# Full coverage (Mac: KMP_DUPLICATE_LIB_OK=TRUE, --max-train-buckets 1500)  
KMP_DUPLICATE_LIB_OK=TRUE python3 scripts/run_tgcn_time_multiseed.py --edges artifacts/edges_2012_2019_  
  
# Candidate scoring (full coverage, chunked for memory)  
KMP_DUPLICATE_LIB_OK=TRUE python3 scripts/score_tgcn_candidates_chunked.py --edges artifacts/edges_2012_  
  
# Run baseline (capped, for laptops) - best tuned config: emb32, ep5, lr0.001  
python3 scripts/run_tgcn_time_multiseed.py --edges artifacts/edges_2012_2019_cap5000_even30.parquet --e  
  
# Rolling-window cross-validation (5 folds, mean ± std across folds)  
python3 scripts/run_tgcn_time_multiseed.py --edges artifacts/edges_2012_2019_cap5000_even30.parquet --e  
  
# Hyperparameter tuning  
python3 scripts/tune_tgcn_hyperparams.py --edges artifacts/edges_2012_2019_cap5000_even30.parquet --out  
  
# Overlap validation (use --max-files-per-year 100 for better coverage)  
python3 scripts/compute_pair_overlap_from_daily.py --pairs artifacts/tgcn_candidates_fullcoverage_top20  
  
# Generate case study plots (use case_study_pairs.csv for the 4 presentation pairs)  
python3 scripts/compute_pair_overlap_from_daily.py --pairs artifacts/case_study_pairs.csv --daily-root
```